атрибут.

Автозаполнение характеристик товаров

Продуктовый бриф. Кейс М.Видео, задача 3

Команда: «Большой Куш»
Участники: Евлоев Аслан, Дзортов Мухамед

1 ВОПРОС 1

Какую задачу бизнеса решаем и кто наш пользователь

Задача

Продавцы на маркетплейсе заполняют карточки как попало. Цвет, память или диагональ часто есть только в описании, и каждый пишет их по-своему. Если поле не заполнено, товар не попадает в фильтр, и покупатель его просто не находит. Меньше находят, значит меньше покупают.

Кто пользователь

1) **Команда каталога М.Видео.** Запускает сервис на всех карточках и разбирает только спорные значения.
2) **Покупатель.** Находит товар через фильтры и может сравнить похожие.
3) **Продавец.** Не тратит время на ручное заполнение каждого поля.

546 тыс.

товаров в каталоге, который нам дали

44%

товаров вообще имеют описание, у остальных есть только название

1635

разных способов записать цвет нашли в данных

2 ВОПРОС 2

Какие требования и ограничения кейса мы учитываем

1) **На входе только текст:** название, описание и поля продавца. Фото не используем.
2) **Приводим к одному виду.** «Black» и «чёрный» становятся «черный», а 1 ТБ становится 1024 ГБ.
3) **Ничего не выдумываем.** У каждого значения есть цитата из текста карточки. Нет в тексте, значит поле пустое.
4) **Ставим уверенность.** Ниже порога значение в каталог сам не попадёт.
5) **Дёшево.** Работает на обычном компьютере, без видеокарты и без платных API.
6) **Честно для всех.** Редкие категории и маленьких продавцов проверяем отдельно.

Какие поля достаём. Для всех категорий бренд, модель и цвет. Для смартфонов ещё память, диагональ и батарею, для ноутбуков память, накопитель, диагональ и процессор. Это как раз обязательные поля из условия кейса.

3 ВОПРОС 3

Как устроен основной сценарий

1
Карточка
Приходят название, описание и поля продавца

2
Правила и словари
Находят варианты: бренды, память, диагональ, цвет

3
Обученная модель
Выбирает лучший вариант для модели товара и цвета

4
Проверка
Значение должно быть в тексте, ищем противоречия

5
Результат
JSON: значение, уверенность, цитата и статус

а) Принято

Уверенность выше порога. Значение сразу идёт в карточку.

б) Есть сомнение

Мало уверенности или текст спорит сам с собой. Уходит человеку на проверку.

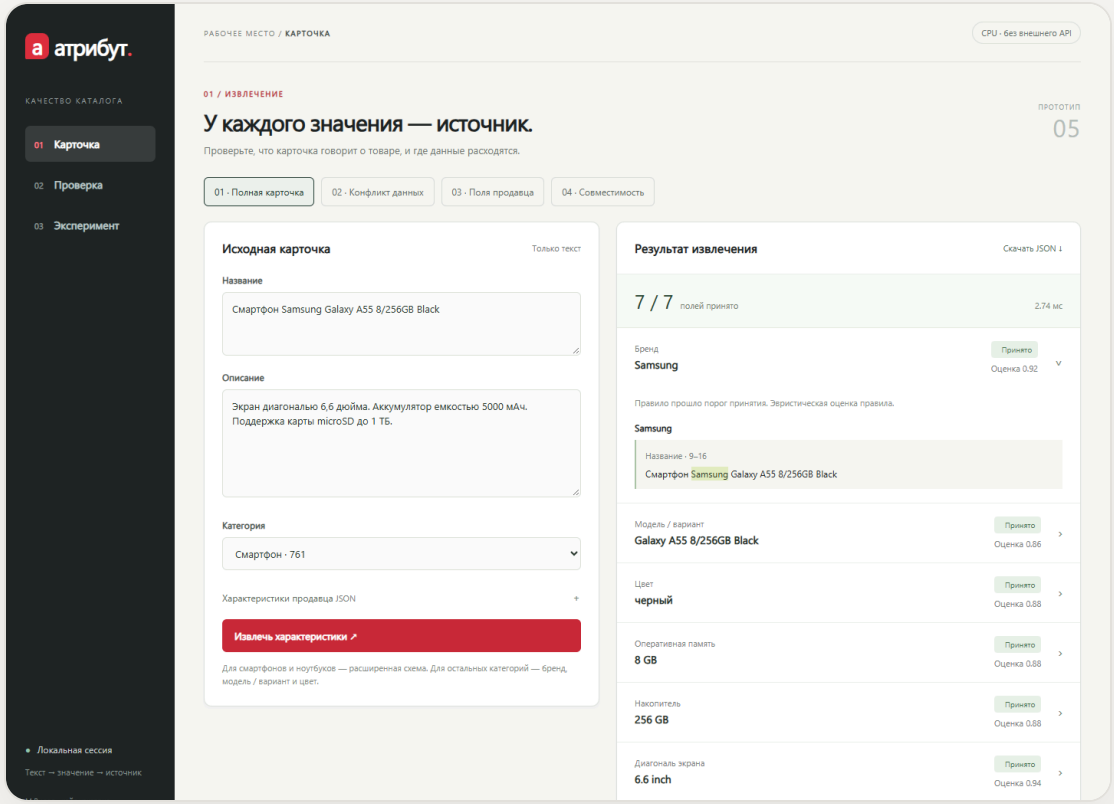
в) Не найдено

В тексте этого нет. Поле остаётся пустым, ничего не придумываем.

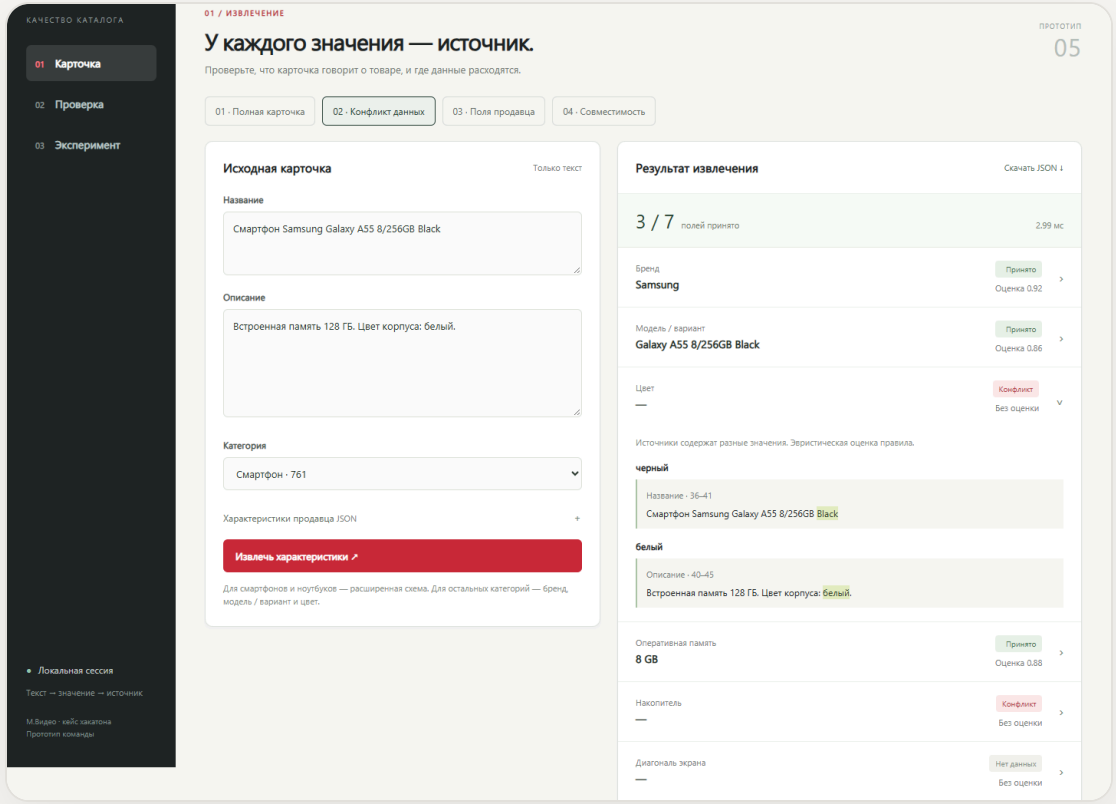
Решения человека сохраняются в журнал, потом по ним пополняем словари и дообучаем модель. Так со временем спорных случаев становится меньше.

Планируем разворачивать в Docker контейнере: внутри Python, модель и интерфейс проверки. Запуск одной командой на любом сервере, видеокарта не нужна. Скорость около 250 карточек в секунду.

Прототип: что видит команда каталога

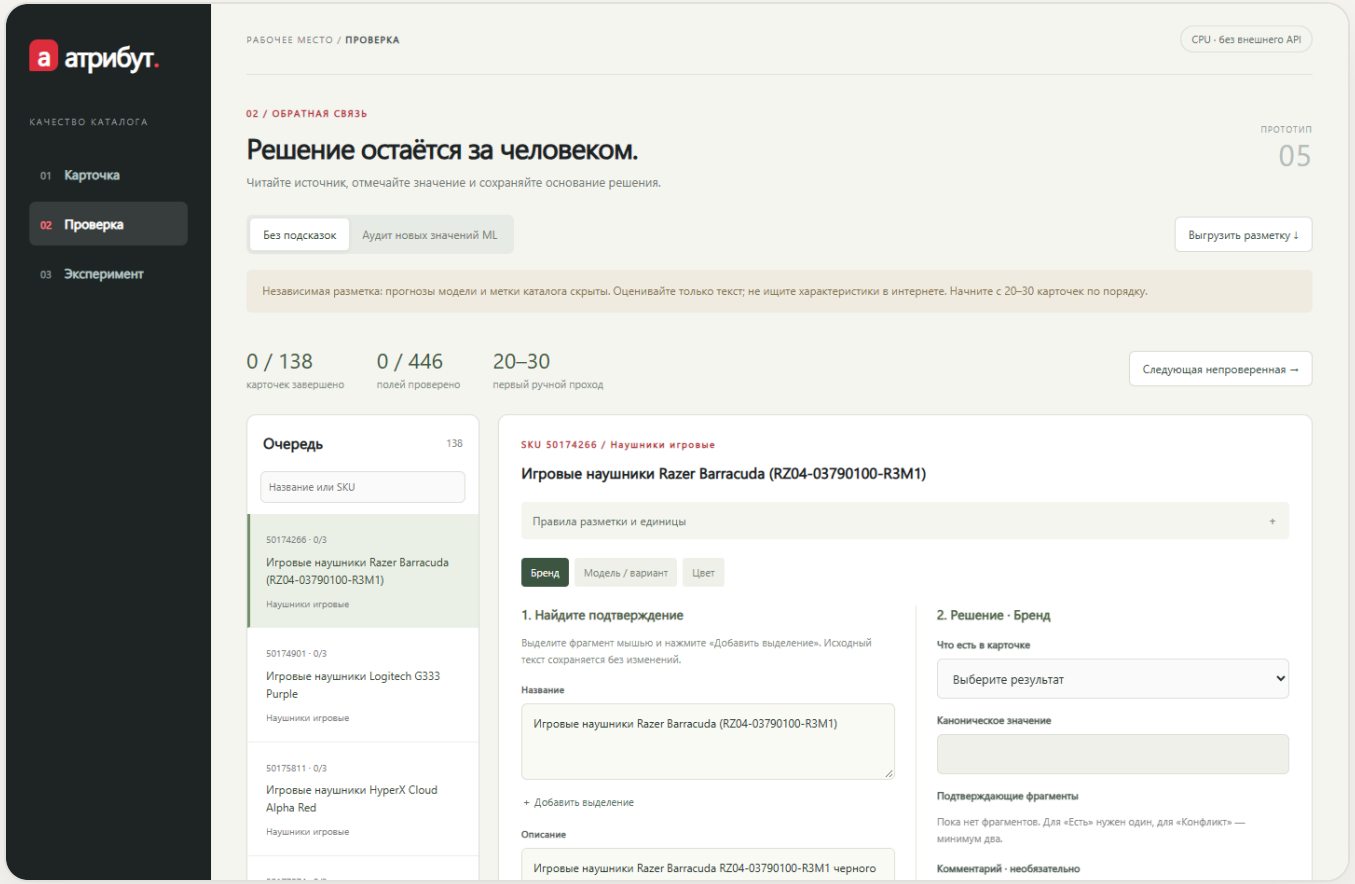


1) Вставляем карточку и жмём «Извлечь». Любое значение можно раскрыть и увидеть, из какого места текста оно взято.



2) В названии «Black», а в описании «белый». Система не выбирает наугад, а помечает конфликт и показывает оба места.

Прототип: ручная проверка



1) Человек читает текст, выбирает ответ и выделяет фрагмент, на который опирается.
2) Подсказки модели скрыты, чтобы не влиять на ответ.
3) Каждое решение пишется в журнал, потом по нему дообучаем модель.

Сейчас на этом экране мы проверяем качество модели. В продукте сюда же пойдут спорные значения.

4 ВОПРОС 4

Какую ценность получают пользователь и бизнес

866 тыс.

значений нашёл сервис на всём каталоге, у каждого есть цитата из текста

около 3%

найденных значений уходят человеку на проверку, это 28 тыс.

26 минут

ушло на весь каталог из 546 тыс. товаров на обычном ПК

Бизнесу

1) Больше товаров попадает в фильтры, их чаще находят и покупают.
2) Команда каталога смотрит только спорное, а не всё подряд.
3) Нет платы за API за каждую карточку.

Покупателю и продавцу

1) Покупатель находит товар по фильтру и может нормально сравнить два похожих.
2) Продавец не заполняет поля руками, а товар всё равно виден в поиске.

Пример из теста: у 4132 товаров из 23 468 мы нашли цвет с цитатой из текста, хотя в каталоге его не было или он был записан так, что фильтр его не понимал.

5 ВОПРОС 5

По каким метрикам поймём, что решение работает

ПОЛЕ	ТОЧНОСТЬ	ПОЛНОТА	ПОКРЫТИЕ
Бренд	98,5%	81,5%	82,7%
Цвет	**93,8%**	**44,1%**	34,4%
Оперативная память	99,0%	77,8%	67,3%
Накопитель	100%	70,4%	70,4%
Диагональ	95,1%	50,3%	52,7%
Батарея	97,1%	**36,5%**	32,7%
Процессор	100%	72,4%	47,1%

Тест на 23 468 карточках, которые мы не трогали, пока делали решение. Сравниваем с данными каталога. Проверку людьми по категориям готовим следующим шагом.

1) **Точность и полнота по каждому полю,** а не одно общее число, чтобы не спрятать провалы.
2) **Покрытие:** в какой доле карточек поле стало заполнено.
3) **Срезы:** отдельно редкие категории и маленькие продавцы.
4) **Спрос:** популярные товары весят больше.

Для нас решение работает, если точность по каждому полю не ниже 95%, а покрытие растёт. По цвету точность пока чуть ниже, 93,8%. А цвет и батарею мы находим реже чем в половине случаев. Это будем улучшать первым делом.